

Ideas Worth Discussing: Final List

Group 1: Continuing Work Already Started

Open threads from our existing work.

1. **Remove more than one piece at once.** Every test so far has removed one part of Panda at a time and found nothing conclusive. If the mechanism only shows up when two or three pieces are removed together, no one-at-a-time test would catch it. Worth designing the cheapest version of a multi-part removal test.
2. **Find a real explanation for why the lift helps on some systems and hurts on others.** The lift helps on calmer systems like Burgers and hurts on genuinely chaotic ones, but our first explanation for why was wrong. Worth working out a better one before running anything new.
3. **Strengthen the rollout finding.** Panda's longer forecasts hold their shape better than Chronos's, based on only two systems, each tested once. Worth adding more evidence before building anything else on top of it.
4. **Pick the first small test on real weather data.** Every real dataset tested is non-chaotic, and every chaotic system tested is simulated. Real weather data would test the original question on something that's both. Worth agreeing on the smallest first experiment rather than a full evaluation.
5. **Build the “pick the better model automatically” tool.** Train a small system that looks at the data and decides whether to trust Panda or Chronos, without needing to know why either model works. Cheap and practical.
6. **Directly test whether the “cycles per window” pattern comes from patch size.** Panda's accuracy oscillates depending on how many repeating cycles fit inside its input window, and the pattern lines up with patch size. Worth changing the patch size directly and checking whether the pattern moves with it.
7. **Re-check the lift's math one system at a time instead of pooled.** The lift's inner math was tested with one shared rule fit across all systems at once, which may have unfairly penalized Burgers. Refitting separately per system would settle this.

Group 2: Borrowing Pieces Between Panda and Other Models

8. **Split the lift into its two halves and test them separately.** The lift combines two different kinds of features, never tested apart. Testing each half alone and both together, matched in size, would show which half is doing the work.

9. **Give MOMENT the ability to look across channels.** MOMENT processes each channel separately. Adding a cross-channel mechanism and testing it the same way as the lift transplant would check whether Panda’s own channel-attention null holds in a different architecture.
10. **Just change MOMENT’s patch size, nothing else.** Keep MOMENT unchanged except its patch length, set to match Panda’s. Cheap, isolates one variable, connects to idea #15.
11. **Train MOMENT to forecast from the start, instead of fill-in-the-blank first.** MOMENT is pretrained on masked reconstruction, then adapted to forecasting afterward. Training it to forecast directly from the start, the way Panda does, is something MOMENT’s own authors flagged as worth testing and never did.
12. **Combine phase-based chunking with the lift.** PhaseFormer chunks signals by wave-cycle position instead of fixed length. Feeding phase-based chunks into the lift may fit its wave-based design better than Panda’s current raw chunks.
13. **Combine explicit cycle-removal with the lift.** CycleNet removes the repeating part of a signal before forecasting what’s left. Testing the lift on that leftover part would show whether its value is about handling everything, or just the non-repeating part.

Group 3: Some New Ideas

14. **Test why the lift helps the simplest system and hurts the most complex ones.** The lift is the same size regardless of what it’s processing, and the pattern runs backwards from what you’d expect: it helps on Burgers, the calmest system tested, and hurts on the genuinely chaotic ones. A version whose size scales with system complexity would test whether that mismatch, or something else, explains the reversal.
15. **Check whether Panda’s chunk size actually fits how complex each system is.** There’s a rule for how large an input chunk needs to be, based on a system’s dimensionality. Free re-analysis: check whether the systems where the lift struggled are the ones where the chunk size was too small.
16. **Check whether the rollout advantage fades in a predictable way.** As a rollout progresses, more of what the model sees is its own past guesses, not real data. Worth checking whether the shape-preserving advantage shrinks at a steady rate or holds up evenly.
17. **Check how much the lift’s result depends on luck of the draw.** The lift’s numbers are randomly fixed once. Rerunning with a different random draw would show whether results are stable or highly seed-dependent, before deciding trainability is the right fix.
18. **Build a better way to measure “how similar is this new system to what Panda was trained on.”** The earlier attempt broke on chaotic systems because it needed matching repeating cycles. A method based on roughness or unpredictability instead of cycles might work where the first one failed.
19. **Check whether Panda’s internal representations alone can tell chaotic and non-**

chaotic systems apart. Freeze Panda, extract its internal representations for a range of signals, and check whether a simple probe can separate chaotic from non-chaotic using only those representations. Cheap, no retraining.

Group 4: Working It Out On Paper First

Each of these can be worked out on paper before any code is written, producing a testable prediction.

20. **Work out what the lift's features should be, and resolve a real contradiction.** Burgers is the one system where the lift helps forecasting the most, yet it's also the worst-fitting case for the lift's own underlying theory. Working out what genuinely theory-correct features would look like for Burgers, and testing them, would show whether the current benefit has anything to do with the theory at all, or comes from somewhere else entirely.
21. **Work out whether Panda's chunk size is too big, not just too small.** The existing rule only gives a minimum chunk size; it says nothing about an upper limit. Established methods exist for finding the smallest sufficient chunk size per system. Applying them would show whether Panda's fixed size is undersized, about right, or oversized for each system tested.
22. **Check whether the lift simply has too few features to work properly.** There's theory on how many random features are needed for a given level of complexity. Checking Panda's actual feature count against that theory could reveal a simpler explanation than fixed versus trainable: just not enough features.
23. **Ask whether Panda's advantage scales with how unpredictable a system is.** There's a known relationship between how chaotic a system is and how fast it generates new information. Checking whether Panda's advantage scales with that rate, using data already collected, would be a more precise claim than anything said so far.
24. **Check whether Panda struggles on systems with symmetries.** Lorenz's z-channel is weak because of a mirror symmetry it can't distinguish. Whether this generalizes, and whether Panda's failures line up with symmetry mismatches elsewhere, has never been checked.